

Proyectos

Pepe Calderón, Fernando Sola, Daniel Ayala, Ana Belén Sánchez, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz



Escuela Técnica Superior de
Ingeniería Informática

1. Diagrama de clases	3
2. Intensión	4
3. Álgebra relacional	6
4. Script SQL para crear la base de datos	10
5. Script SQL para la carga inicial de datos	13
6. Triggers de integridad	16
7. Consultas	18
8. Pruebas SQL	21

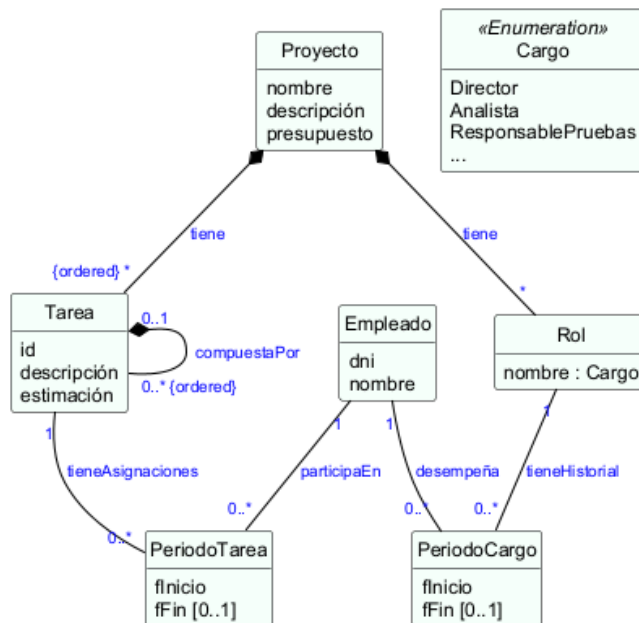
Requisitos La transcripción que aparece a continuación corresponde a una entrevista a una ingeniera de software que necesita un sistema de información para ayudarle en la gestión de sus proyectos.

- Pregunta: Bien, coménteme cuál sería el principal objetivo del sistema que usted necesita.
- Respuesta: Bueno, básicamente lo que quiero es un gestor de proyectos sencillo, que me permita gestionar las tareas asociadas y asignárselas a los empleados.
- P: Empecemos entonces por los proyectos, ¿qué información quiere que gestione el sistema sobre ellos?
- R: Básicamente, el nombre del proyecto, una descripción y el presupuesto que tiene. Puede que luego sea interesante añadirle más información, pero de momento me valdría con eso.
- P: De acuerdo, ha hablado antes de tareas. ¿Me puede explicar ese concepto?
- R: Sí, claro. Nosotros descomponemos los proyectos en tareas, lo que se conoce como WBS (Work Breakdown Structure). Cada tarea tiene un identificador que usamos para referirnos a ella, p.e. la tarea T-28F. También tienen una descripción y una estimación del coste de su realización. En el caso de que una tarea sea muy compleja, puede descomponerse en subtareas, que también pueden descomponerse recursivamente.
- P: Entiendo, ¿esas tareas tienen algún orden especial?
- R: El que le vayamos dando al crearlas, aunque luego podríamos cambiarlo.
- P: Entiendo, un proyecto tiene una secuencia de tareas que a su vez pueden tener una secuencia de subtareas y así recursivamente. Debemos conocer el orden. ¿Qué más hay que saber sobre las tareas?
- R: Pues que una vez que creamos la WBS, vamos asignando las tareas a los empleados.
- P: ¿En qué consiste una asignación de una tarea a un empleado?
- R: Básicamente, se encarga a un empleado la realización de una tarea con una fecha de inicio y otra de fin.
- P: ¿A cuántos empleados se le asigna una misma tarea?
- R: Cuando decidimos a quién asignársela, a uno sólo. Nuestras tareas simples están pensadas para que las pueda realizar un solo empleado.
- P: ¿Qué es una tarea simple?
- R: Una tarea que no está descompuesta en tareas más simples.
- P: Entiendo, sólo se asignan tareas simples, ¿no es así?
- R: Bueno, no necesariamente. Podemos asignar tareas complejas para su supervisión.
- P: Entiendo, se puede asignar cualquier tarea (compleja o simple) a cualquier empleado.
- R: Sí, aunque eso depende del rol que tenga el empleado en el proyecto.
- P: Explíqueme eso.
- R: Bueno, en cada proyecto se definen una serie de roles y cada empleado puede desempeñar distintos roles en distintos proyectos. Por ejemplo, uno puede ser el director de un proyecto y a la vez ser el responsable de pruebas de otro proyecto.
- P: Entiendo, ¿un mismo empleado puede jugar más de un rol en un mismo proyecto?
- R: No es frecuente, pero podría ocurrir, especialmente en proyectos pequeños.
- P: Una vez establecido un rol de un empleado en un proyecto, ¿puede cambiar?
- R: Sí, reasignamos los empleados según los proyectos que van saliendo.
- P: Entonces, le interesa saber desde qué fecha hasta qué fecha un empleado ha desempeñado un rol en un proyecto ¿no?

- R: Sí, quiero saber qué empleados han ido pasando por cada proyecto y en qué fechas.
- P: Entiendo que cada rol está asociado a un cargo predefinido, ¿es así?
- R: Sí, claro. Los nombres de los roles coinciden con los cargos habituales: director, analista, responsable de pruebas, etc. Tenemos definidos los roles según la norma ISO-9001.
- P: Muy bien, creo que con eso tengo para una primera versión. Gracias.

Modelo Conceptual

Diagrama de clases



RN-01: La descripción del proyecto es obligatoria y las referencias obligatorias a proyecto, tarea, rol o empleado no admiten NULL.
 RN-02: Los periodos permiten historial; fFin puede quedar sin definir, pero si existe no puede ser anterior a fInicio. Las fechas son inclusivas.
 RN-03: Los periodos de una tarea no se solapan; solo hay una persona responsable activa y se permiten reasignaciones históricas.
 RN-04: Una subtarea pertenece al mismo proyecto que su tarea padre.
 RN-05: Varios empleados pueden desempeñar el mismo rol simultáneamente.

Modelo Relacional

Intensión

```

Proyectos = { proyectoId, nombre, descripcion, presupuesto }
  PK(proyectoId)
Roles = { rolId, proyectoId, nombre }
  PK(rolId)
  FK(proyectoId) / Proyectos
Tareas = { tareaId, proyectoId, orden, id, descripcion, estimacion }
  PK(tareaId)
  FK(proyectoId) / Proyectos
  AK(proyectoId, orden)
Subtareas = { subtareaId, tareaId, orden }
  PK(subtareaId)
  FK(subtareaId) / Tareas
  FK(tareaId) / Tareas
  AK(tareaId, orden)
Empleados = { empleadoId, dni, nombre }
  PK(empleadoId)
  AK(dni)
PeriodosCargos = { periodoCargoId, empleadoId, rolId, fInicio, fFin }
  PK(periodoCargoId)
  FK(empleadoId) / Empleados
  FK(rolId) / Roles
PeriodosTareas = { periodoTareaId, empleadoId, tareaId, fInicio, fFin }
  PK(periodoTareaId)
  FK(empleadoId) / Empleados
  FK(tareaId) / Tareas

Proyectos = {
  (1, "Portal de alquileres", "Plataforma de alquiler de viviendas", 50000.00),
  (2, "Gestor interno", "Herramientas internas del equipo", 90000.00)
}

Roles = {
  (r1, 1, "Director"),
  (r2, 1, "Analista"),

```

```

    (r3, 2, "Responsable de pruebas")
}

Tareas = {
    (t1, 1, 1, "T-01", "Diseño del modelo", 40),
    (t2, 1, 2, "T-02", "Implementación de API", 80),
    (t3, 2, 1, "T-10", "Plan de pruebas", 60),
    (t4, 1, 3, "T-03", "Pruebas unitarias", 30),
    (t5, 1, 4, "T-04", "Pruebas de integración", 25),
    (t6, 2, 2, "T-11", "Validación del plan de pruebas", 30)
}

Subtareas = {
    (t4, t2, 1),
    (t5, t4, 1),
    (t6, t3, 1)
}

Empleados = {
    (e1, "11111111A", "Ana García"),
    (e2, "22222222B", "Luis Pérez"),
    (e3, "33333333C", "Marta López"),
    (e4, "44332211D", "Ana Rueda")
}

PeriodosCargos = {
    (pc1, e1, r1, "2024-01-01", "2024-01-31"),
    (pc2, e1, r1, "2024-02-01", NULL),
    (pc3, e2, r2, "2024-01-15", NULL),
    (pc4, e3, r3, "2024-02-01", NULL),
    (pc5, e4, r1, "2024-03-01", NULL)
}

PeriodosTareas = {
    (pt1, e2, t1, "2024-02-01", "2024-02-15"),
    (pt2, e1, t1, "2024-02-16", "2024-02-28"),
    (pt3, e2, t1, "2024-03-01", "2024-03-15"),
    (pt4, e2, t2, "2024-02-16", NULL),
    (pt5, e3, t3, "2024-03-01", NULL),
    (pt6, e2, t4, "2024-02-20", NULL)
}

```

RN-01: La descripción del proyecto y las referencias obligatorias no admiten valores nulos.

RN-02: Los periodos pueden repetirse en el historial; su fecha de fin es opcional y, si existe, no precede a la fecha de inicio; las fechas son inclusivas. RN-03: Los periodos de una tarea no se solapan. RN-04: Cada subtarea pertenece al mismo proyecto que su tarea padre. RN-05: Varios empleados pueden desempeñar el mismo rol simultáneamente.

Álgebra relacional

- Renombrado:

$$P \leftarrow \begin{matrix} \rho \\ P(pid,pn,pdesc,pres) \end{matrix} (Proyectos)$$

$$R \leftarrow \begin{matrix} \rho \\ R(rid,pid,rn) \end{matrix} (Roles)$$

$$T \leftarrow \begin{matrix} \rho \\ T(tid,pid,ord,cod,tdesc,est) \end{matrix} (Tareas)$$

$$E \leftarrow \begin{matrix} \rho \\ E(eid,dni,en) \end{matrix} (Empleados)$$

$$PC \leftarrow \begin{matrix} \rho \\ PC(pcid,eid,rid,fi,ff) \end{matrix} (PeriodosCargos)$$

$$PT \leftarrow \begin{matrix} \rho \\ PT(ptid,eid,tid,fi,ff) \end{matrix} (PeriodosTareas)$$

$$ST \leftarrow \begin{matrix} \rho \\ ST(stid,tid,sord) \end{matrix} (Subtareas)$$

- Empleados con roles en proyectos:

$$ER \leftarrow E \bowtie PC \bowtie R$$

ER = { eid, dni, en, pcid, rid, fi, ff, pid, rn }

ER = {
(e1, "11111111A", "Ana García", pc1, r1, "2024-01-01", "2024-01-31", 1, "Director"),

```
{
  (e1, "11111111A", "Ana García", pc2, r1, "2024-02-01", NULL, 1, "Director"),
  (e2, "22222222B", "Luis Pérez", pc3, r2, "2024-01-15", NULL, 1, "Analista"),
  (e3, "33333333C", "Marta López", pc4, r3, "2024-02-01", NULL, 2, "Responsable de pruebas"),
  (e4, "44332211D", "Ana Rueda", pc5, r1, "2024-03-01", NULL, 1, "Director")
}
```

- Empleados con tareas en proyectos:

$$ET \leftarrow E \bowtie PT \bowtie T$$

```
ET = { eid, dni, en, ptid, tid, fi, ff, pid, ord, cod, tdesc, est }

ET = {
  (e2, "22222222B", "Luis Pérez", pt1, t1, "2024-02-01", "2024-02-15", 1, 1, "T-01", "Diseño del modelo", 40),
  (e1, "11111111A", "Ana García", pt2, t1, "2024-02-16", "2024-02-28", 1, 1, "T-01", "Diseño del modelo", 40),
  (e2, "22222222B", "Luis Pérez", pt3, t1, "2024-03-01", "2024-03-15", 1, 1, "T-01", "Diseño del modelo", 40),
  (e2, "22222222B", "Luis Pérez", pt4, t2, "2024-02-16", NULL, 1, 2, "T-02", "Implementación de API", 80),
  (e3, "33333333C", "Marta López", pt5, t3, "2024-03-01", NULL, 2, 1, "T-10", "Plan de pruebas", 60),
  (e2, "22222222B", "Luis Pérez", pt6, t4, "2024-02-20", NULL, 1, 3, "T-03", "Pruebas unitarias", 30)
}
```

- Empleados que trabajan en proyectos (roles o tareas):

$$EP \leftarrow \Pi_{eid,en,pid}(ER) \cup \Pi_{eid,en,pid}(ET)$$

```
EP = { eid, en, pid }

EP = {
  (e1, "Ana García", 1),
  (e2, "Luis Pérez", 1),
  (e3, "Marta López", 2),
  (e4, "Ana Rueda", 1)
}
```

- Empleados que trabajan en proyectos (roles y tareas):

$$EP \leftarrow \Pi_{eid,en,pid}(ER) \cap \Pi_{eid,en,pid}(ET)$$

```
EP = { eid, en, pid }

EP = {
  (e1, "Ana García", 1),
  (e2, "Luis Pérez", 1),
  (e3, "Marta López", 2)
}
```

- Tareas asignadas a empleados en el proyecto 1:

$$TareasEmpleadoP1 \leftarrow \sigma_{pid=1}(ET)$$

```

TareasEmpleadoP1 = { eid, dni, en, ptid, tid, fi, ff, pid, ord, cod, tdesc, est }

TareasEmpleadoP1 = {
  (e2, "22222222B", "Luis Pérez", pt1, t1, "2024-02-01", "2024-02-15", 1, 1, "T-01", "Diseño del modelo", 40),
  (e1, "11111111A", "Ana García", pt2, t1, "2024-02-16", "2024-02-28", 1, 1, "T-01", "Diseño del modelo", 40),
  (e2, "22222222B", "Luis Pérez", pt3, t1, "2024-03-01", "2024-03-15", 1, 1, "T-01", "Diseño del modelo", 40),
  (e2, "22222222B", "Luis Pérez", pt4, t2, "2024-02-16", NULL, 1, 2, "T-02", "Implementación de API", 80),
  (e2, "22222222B", "Luis Pérez", pt6, t4, "2024-02-20", NULL, 1, 3, "T-03", "Pruebas unitarias", 30)
}

```

- Número de tareas por empleado (proyecto 1):

$$NumTareasEmpleadoP1 \leftarrow \gamma_{eid, en, \rho_{total}}^{(COUNT(*))} eid, en(TareasEmpleadoP1)$$

```

NumTareasEmpleadoP1 = { eid, en, total }

NumTareasEmpleadoP1 = {
  (e1, "Ana García", 1),
  (e2, "Luis Pérez", 4)
}

```

- Listado de roles por proyecto:

$$ProyectosRoles \leftarrow P \bowtie R$$

```

ProyectosRoles = { pid, pn, pdesc, pres, rid, rn }

ProyectosRoles = {
  (1, "Portal de alquileres", "Plataforma de alquiler de viviendas", 50000.00, r1, "Director"),
  (1, "Portal de alquileres", "Plataforma de alquiler de viviendas", 50000.00, r2, "Analista"),
  (2, "Gestor interno", "Herramientas internas del equipo", 90000.00, r3, "Responsable de pruebas")
}

```

- Listado de tareas por proyecto:

$$ProyectosTareas \leftarrow P \bowtie T$$

```

ProyectosTareas = { pid, pn, pdesc, pres, tid, ord, cod, tdesc, est }

ProyectosTareas = {
  (1, "Portal de alquileres", "Plataforma de alquiler de viviendas", 50000.00, t1, 1, "T-01", "Diseño del modelo", 40),
  (1, "Portal de alquileres", "Plataforma de alquiler de viviendas", 50000.00, t2, 2, "T-02", "Implementación de API", 80),
  (2, "Gestor interno", "Herramientas internas del equipo", 90000.00, t3, 1, "T-10", "Plan de pruebas", 60),
  (1, "Portal de alquileres", "Plataforma de alquiler de viviendas", 50000.00, t4, 3, "T-03", "Pruebas unitarias", 30),
}

```

3. Álgebra relacional

```
(1, "Portal de alquileres", "Plataforma de alquiler de viviendas", 50000.00, t5, 4, "T-04",  
"Pruebas de integración", 25),  
(2, "Gestor interno", "Herramientas internas del equipo", 90000.00, t6, 2, "T-11",  
"Validación del plan de pruebas", 30)  
}
```

- Número de subtareas por tarea:

$$NumSubtareas \leftarrow \gamma_{tid, \rho_{total}(COUNT(*))} tid(T \bowtie ST)$$

```
NumSubtareas = { tid, total }
```

```
NumSubtareas = {  
  (t2, 1),  
  (t4, 1),  
  (t3, 1)  
}
```

- Empleados con subtareas:

$$EST \leftarrow ET \bowtie ST$$

```
EST = { eid, dni, en, ptid, tid, fi, ff, pid, ord, cod, tdesc, est, stid, sord }
```

```
EST = {  
  (e2, "22222222B", "Luis Pérez", pt4, t2, "2024-02-16", NULL, 1, 2, "T-02", "Implementación  
de API", 80, t4, 1),  
  (e2, "22222222B", "Luis Pérez", pt6, t4, "2024-02-20", NULL, 1, 3, "T-03", "Pruebas  
unitarias", 30, t5, 1),  
  (e3, "33333333C", "Marta López", pt5, t3, "2024-03-01", NULL, 2, 1, "T-10", "Plan de  
pruebas", 60, t6, 1)  
}
```

Modelo Tecnológico

Script SQL para crear la base de datos

proyectos/createDB.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2024
-- Descripción: script para crear el esquema de la BD
--

-- Crear la base de datos
DROP DATABASE IF EXISTS ProyectosDB;
CREATE DATABASE ProyectosDB;
USE ProyectosDB;

-- Crear la tabla proyectos
CREATE OR REPLACE TABLE projects (
    project_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    budget DECIMAL(15, 2) NOT NULL,
    CONSTRAINT ck_projects_budget CHECK (budget > 0)
);

-- Crear la tabla roles
CREATE OR REPLACE TABLE roles (
    role_id INT AUTO_INCREMENT PRIMARY KEY,
```

```

    project_id INT NOT NULL,
    name VARCHAR(255) NOT NULL,
    FOREIGN KEY (project_id) REFERENCES projects(project_id)
);

-- Crear la tabla tareas
CREATE OR REPLACE TABLE tasks (
    task_id INT AUTO_INCREMENT PRIMARY KEY,
    project_id INT NOT NULL,
    position INT NOT NULL,
    task_code CHAR(12) NOT NULL,
    description TEXT NOT NULL,
    estimate INT NOT NULL,
    FOREIGN KEY (project_id) REFERENCES projects(project_id),
    CONSTRAINT uq_tasks_project_position UNIQUE (project_id, position),
    CONSTRAINT uq_tasks_project_code UNIQUE (project_id, task_code),
    CONSTRAINT ck_tasks_position CHECK (position > 0),
    CONSTRAINT ck_tasks_estimate CHECK (estimate > 0)
);

-- Crear la tabla subtareas
CREATE OR REPLACE TABLE subtasks (
    subtask_id INT,
    task_id INT NOT NULL,
    position INT NOT NULL,
    PRIMARY KEY (subtask_id),
    FOREIGN KEY (subtask_id) REFERENCES tasks(task_id),
    FOREIGN KEY (task_id) REFERENCES tasks(task_id),
    CONSTRAINT uq_subtasks_task_position UNIQUE (task_id, position),
    CONSTRAINT ck_subtasks_position CHECK (position > 0)
);

-- Crear la tabla empleados
CREATE OR REPLACE TABLE employees (
    employee_id INT AUTO_INCREMENT PRIMARY KEY,
    national_id VARCHAR(20) NOT NULL UNIQUE,
    name VARCHAR(255) NOT NULL
);

-- Crear la tabla role_periods
CREATE OR REPLACE TABLE role_periods (
    role_period_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT NOT NULL,
    role_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id),
    FOREIGN KEY (role_id) REFERENCES roles(role_id),
    CONSTRAINT ck_role_periods_dates CHECK (end_date IS NULL OR end_date >=
start_date)

```

```
);  
  
-- Crear la tabla task_periods  
CREATE OR REPLACE TABLE task_periods (  
    task_period_id INT AUTO_INCREMENT PRIMARY KEY,  
    employee_id INT NOT NULL,  
    task_id INT NOT NULL,  
    start_date DATE NOT NULL,  
    end_date DATE,  
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id),  
    FOREIGN KEY (task_id) REFERENCES tasks(task_id),  
    CONSTRAINT ck_task_periods_dates CHECK (end_date IS NULL OR end_date >=  
start_date)  
);
```

Script SQL para la carga inicial de datos

proyectos/populateDB.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Procedimiento para poblar la BD de proyectos
--

USE ProyectosDB;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_populate()
BEGIN
    SET FOREIGN_KEY_CHECKS = 0;
    DELETE FROM task_periods;
    DELETE FROM role_periods;
    DELETE FROM subtasks;
    DELETE FROM tasks;
    DELETE FROM roles;
    DELETE FROM employees;
    DELETE FROM projects;
    SET FOREIGN_KEY_CHECKS = 1;

    INSERT INTO projects (project_id, name, description, budget) VALUES
        (1, 'Portal de alquileres', 'Plataforma de alquiler de viviendas',
```

```

50000),
    (2, 'Gestor interno', 'Herramientas internas del equipo', 90000)
;

-- Insertar datos de ejemplo en la tabla roles
INSERT INTO roles (role_id, project_id, name) VALUES
    (1, 1, 'Director'),
    (2, 1, 'Analista'),
    (3, 2, 'Responsable de pruebas')
;

-- Insertar datos de ejemplo en la tabla tareas
INSERT INTO tasks (task_id, project_id, position, task_code,
description, estimate) VALUES
    (1, 1, 1, 'T-01', 'Diseño del modelo', 40),
    (2, 1, 2, 'T-02', 'Implementación de API', 80),
    (3, 2, 1, 'T-10', 'Plan de pruebas', 60),
    (4, 1, 3, 'T-03', 'Pruebas unitarias', 30),
    (5, 1, 4, 'T-04', 'Pruebas de integración', 25),
    (6, 2, 2, 'T-11', 'Validación del plan de pruebas', 30)
;

-- Insertar datos de ejemplo en la tabla subtareas
INSERT INTO subtasks (subtask_id, task_id, position) VALUES
    (4, 2, 1),
    (5, 4, 1),
    (6, 3, 1)
;

-- Insertar datos de ejemplo en la tabla empleados
INSERT INTO employees (employee_id, national_id, name) VALUES
    (1, '11111111A', 'Ana García'),
    (2, '22222222B', 'Luis Pérez'),
    (3, '33333333C', 'Marta López'),
    (4, '44332211D', 'Ana Rueda')
;

-- Insertar datos de ejemplo en la tabla role_periods
INSERT INTO role_periods (role_period_id, employee_id, role_id,
start_date, end_date) VALUES
    (1, 1, 1, '2024-01-01', '2024-01-31'),
    (2, 1, 1, '2024-02-01', null),
    (3, 2, 2, '2024-01-15', null),
    (4, 3, 3, '2024-02-01', null),
    (5, 4, 1, '2024-03-01', null)
;

-- Insertar datos de ejemplo en la tabla task_periods
INSERT INTO task_periods (task_period_id, employee_id, task_id,
start_date, end_date) VALUES

```

```
(1, 2, 1, '2024-02-01', '2024-02-15'),  
(2, 1, 1, '2024-02-16', '2024-02-28'),  
(3, 2, 1, '2024-03-01', '2024-03-15'),  
(4, 2, 2, '2024-02-16', null),  
(5, 3, 3, '2024-03-01', null),  
(6, 2, 4, '2024-02-20', null)  
;  
  
END //  
DELIMITER ;  
  
CALL p_populate();
```

Triggers de integridad

proyectos/triggers.sql

```
--
-- Triggers para las invariantes temporales y jerárquicas de ProyectosDB
--
USE ProyectosDB;

DELIMITER //
CREATE OR REPLACE TRIGGER t_biu_task_periods_no_overlap
BEFORE INSERT OR UPDATE ON task_periods
FOR EACH ROW
BEGIN
    DECLARE v_overlapping_periods INT DEFAULT 0;

    SELECT COUNT(*) INTO v_overlapping_periods
    FROM task_periods existing_period
    WHERE existing_period.task_id = NEW.task_id
        AND existing_period.task_period_id <> COALESCE(NEW.task_period_id, 0)
        AND (existing_period.end_date IS NULL OR NEW.start_date <=
existing_period.end_date)
        AND (NEW.end_date IS NULL OR existing_period.start_date <=
NEW.end_date);

    IF v_overlapping_periods > 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'RN-03: Los periodos de una tarea no pueden
solaparse';
    END IF;
END IF;
```

```
END //
```



```
CREATE OR REPLACE TRIGGER t_biu_subtasks_same_project
BEFORE INSERT OR UPDATE ON subtasks
FOR EACH ROW
BEGIN
    DECLARE v_same_project INT DEFAULT 0;

    SELECT COUNT(*) INTO v_same_project
    FROM tasks child
    JOIN tasks parent ON parent.project_id = child.project_id
    WHERE child.task_id = NEW.subtask_id
        AND parent.task_id = NEW.task_id;

    IF v_same_project = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'RN-04: La tarea y su subtarea deben
pertener al mismo proyecto';
    END IF;
END //
DELIMITER ;
```

Consultas

proyectos/queries.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Consultas SQL correspondientes al álgebra relacional de
-- proyectos

USE ProyectosDB;

-- ER: empleados con roles en proyectos
CREATE OR REPLACE VIEW v_employee_roles AS
SELECT e.employee_id AS eid, e.national_id, e.name AS en,
       pc.role_period_id AS pcid, pc.role_id AS rid,
       pc.start_date AS fi, pc.end_date AS ff,
       r.project_id AS pid, r.name AS rn
FROM employees e
JOIN role_periods pc ON pc.employee_id = e.employee_id
JOIN roles r ON r.role_id = pc.role_id;

SELECT * FROM v_employee_roles;

-- ET: empleados con tareas en proyectos
CREATE OR REPLACE VIEW v_employee_tasks AS
SELECT e.employee_id AS eid, e.national_id, e.name AS en,
       pt.task_period_id AS ptid, pt.task_id AS tid,
       pt.start_date AS fi, pt.end_date AS ff,
```

```

        t.project_id AS pid, t.position AS ord, t.task_code AS cod,
        t.description AS tdesc, t.estimate AS est
FROM employees e
JOIN task_periods pt ON pt.employee_id = e.employee_id
JOIN tasks t ON t.task_id = pt.task_id;

SELECT * FROM v_employee_tasks;

-- EP: empleados que trabajan en proyectos, por rol o por tarea
SELECT eid, en, pid
FROM v_employee_roles
UNION
SELECT eid, en, pid
FROM v_employee_tasks;

-- EP: empleados que trabajan en proyectos, tanto por rol como por tarea
SELECT eid, en, pid
FROM v_employee_roles
INTERSECT
SELECT eid, en, pid
FROM v_employee_tasks;

-- TareasEmpleadoP1: tareas asignadas a empleados en el proyecto 1
SELECT *
FROM v_employee_tasks
WHERE pid = 1;

-- NumTareasEmpleadoP1: Número de tareas por empleado en el proyecto 1
SELECT eid, en, COUNT(*) AS total
FROM v_employee_tasks
WHERE pid = 1
GROUP BY eid, en;

-- ProyectosRoles: roles de cada proyecto
SELECT p.project_id AS pid, p.name AS pn, p.description AS pdesc,
       p.budget AS pres, r.role_id AS rid, r.name AS rn
FROM projects p
JOIN roles r ON r.project_id = p.project_id;

-- ProyectosTareas: tareas de cada proyecto
SELECT p.project_id AS pid, p.name AS pn, p.description AS pdesc, p.budget
AS pres,
       t.task_id AS tid, t.position AS ord, t.task_code AS cod,
       t.description AS tdesc, t.estimate AS est
FROM projects p
JOIN tasks t ON t.project_id = p.project_id;

-- NumSubtareas: Número de subtareas por tarea
SELECT t.task_id AS tid, COUNT(*) AS total
FROM tasks t

```

```
JOIN subtasks st ON st.task_id = t.task_id
GROUP BY t.task_id;

-- EST: empleados con subtareas
SELECT et.*, st.subtask_id AS stid, st.position AS sord
FROM v_employee_tasks et
JOIN subtasks st ON st.task_id = et.tid;
```

Pruebas SQL

proyectos/tests.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Tests negativos para ProyectosDB
--
USE ProyectosDB;

CREATE OR REPLACE TABLE test_results (
    test_id VARCHAR(20) PRIMARY KEY,
    test_name VARCHAR(200) NOT NULL,
    test_message VARCHAR(500) NOT NULL,
    test_status ENUM('PASS', 'FAIL', 'ERROR') NOT NULL,
    execution_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
CREATE OR REPLACE PROCEDURE p_log_test(p_id VARCHAR(20), p_message
VARCHAR(500), p_status VARCHAR(10))
BEGIN
    INSERT INTO test_results VALUES (p_id, SUBSTRING_INDEX(p_message, ':',
1), p_message, p_status, DEFAULT);
END //

CREATE OR REPLACE PROCEDURE p_test_ck01_budget()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('CK01', 'CK01: El
```

```

presupuesto debe ser positivo', 'PASS');
    CALL p_populate();
    INSERT INTO projects (name, budget) VALUES ('Proyecto inválido', 0);
    CALL p_log_test('CK01', 'ERROR: Se permitió un presupuesto no
positivo', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_uq01_task_position()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('UQ01', 'UQ01: El
orden de las tareas es único por proyecto', 'PASS');
    CALL p_populate();
    INSERT INTO tasks (project_id, position, task_code, description,
estimate) VALUES (1, 1, 'T-X', 'Duplicada', 10);
    CALL p_log_test('UQ01', 'ERROR: Se permitió duplicar el orden de una
tarea', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_uq02_task_code()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('UQ02', 'UQ02: El
identificador de tarea es único por proyecto', 'PASS');
    CALL p_populate();
    INSERT INTO tasks (project_id, position, task_code, description,
estimate) VALUES (1, 10, 'T-01', 'Duplicada', 10);
    CALL p_log_test('UQ02', 'ERROR: Se permitió duplicar el identificador
de tarea', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn01_project_description()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN01A', 'RN-01:
La descripción del proyecto es obligatoria', 'PASS');
    CALL p_populate();
    INSERT INTO projects (name, description, budget) VALUES ('Sin
descripción', NULL, 1000);
    CALL p_log_test('RN01A', 'ERROR: Se permitió un proyecto sin
descripción', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn01_role_project()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN01B', 'RN-01:
Todo rol pertenece a un proyecto', 'PASS');
    CALL p_populate();
    INSERT INTO roles (project_id, name) VALUES (NULL, 'Rol sin proyecto');
    CALL p_log_test('RN01B', 'ERROR: Se permitió un rol sin proyecto',
'FAIL');
END //

```

```

CREATE OR REPLACE PROCEDURE p_test_rn01_task_project()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN01C', 'RN-01:
    Toda tarea pertenece a un proyecto', 'PASS');
    CALL p_populate();
    INSERT INTO tasks (project_id, position, task_code, description,
    estimate) VALUES (NULL, 10, 'T-99', 'Sin proyecto', 10);
    CALL p_log_test('RN01C', 'ERROR: Se permitió una tarea sin proyecto',
    'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn01_role_period_employee()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN01D', 'RN-01:
    Todo periodo de rol pertenece a un empleado', 'PASS');
    CALL p_populate();
    INSERT INTO role_periods (employee_id, role_id, start_date) VALUES
    (NULL, 1, '2024-04-01');
    CALL p_log_test('RN01D', 'ERROR: Se permitió un periodo de rol sin
    empleado', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn01_role_period_role()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN01E', 'RN-01:
    Todo periodo de rol refiere un rol', 'PASS');
    CALL p_populate();
    INSERT INTO role_periods (employee_id, role_id, start_date) VALUES (1,
    NULL, '2024-04-01');
    CALL p_log_test('RN01E', 'ERROR: Se permitió un periodo sin rol',
    'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn01_task_period_employee()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN01F', 'RN-01:
    Todo periodo de tarea pertenece a un empleado', 'PASS');
    CALL p_populate();
    INSERT INTO task_periods (employee_id, task_id, start_date) VALUES
    (NULL, 1, '2024-04-01');
    CALL p_log_test('RN01F', 'ERROR: Se permitió un periodo de tarea sin
    empleado', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn01_task_period_task()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN01G', 'RN-01:
    Todo periodo de tarea refiere una tarea', 'PASS');
    CALL p_populate();
    INSERT INTO task_periods (employee_id, task_id, start_date) VALUES (1,

```

```

NULL, '2024-04-01');
    CALL p_log_test('RN01G', 'ERROR: Se permitió un periodo sin tarea',
'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_ck02_period()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('CK02', 'CK02: El
fin de un periodo no puede preceder a su inicio', 'PASS');
    CALL p_populate();
    INSERT INTO task_periods (employee_id, task_id, start_date, end_date)
VALUES (1, 2, '2026-02-01', '2026-01-01');
    CALL p_log_test('CK02', 'ERROR: Se permitió un periodo con fechas
inválidas', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn03_task_period_overlap()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN03', 'RN-03:
Los periodos de una tarea no se solapan', 'PASS');
    CALL p_populate();
    INSERT INTO task_periods (employee_id, task_id, start_date, end_date)
VALUES (3, 1, '2024-02-10', '2024-02-20');
    CALL p_log_test('RN03', 'ERROR: Se permitió solapar periodos de una
tarea', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn03_task_period_update_overlap()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN03B', 'RN-03:
No se puede actualizar un periodo para que se solape', 'PASS');
    CALL p_populate();
    UPDATE task_periods
SET start_date = '2024-02-10', end_date = '2024-02-20'
WHERE task_period_id = 3;
    CALL p_log_test('RN03B', 'ERROR: Se permitió actualizar un periodo
solapado', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn04_subtask_same_project()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN04A', 'RN-04:
Padre y subtarea pertenecen al mismo proyecto', 'PASS');
    CALL p_populate();
    INSERT INTO subtasks (subtask_id, task_id, position) VALUES (1, 3, 2);
    CALL p_log_test('RN04A', 'ERROR: Se vinculó una subtarea de otro
proyecto', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn04_subtask_update_project()

```

```

BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN04B', 'RN-04:
La actualización conserva el proyecto de la WBS', 'PASS');
    CALL p_populate();
    UPDATE subtasks SET task_id = 3 WHERE subtask_id = 4;
    CALL p_log_test('RN04B', 'ERROR: Se movió una subtarea a otro
proyecto', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_uq03_national_id()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('UQ03', 'UQ03: El
DNI de empleado debe ser único', 'PASS');
    CALL p_populate();
    INSERT INTO employees (national_id, name) VALUES ('11111111A',
'Empleado duplicado');
    CALL p_log_test('UQ03', 'ERROR: Se permitió duplicar el DNI', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_run_tests()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        CALL p_log_test('POPULATE', 'ERROR: La carga inicial falló',
'ERROR');
        SELECT * FROM test_results ORDER BY test_id;
    END;
    DELETE FROM test_results;
    CALL p_populate();
    CALL p_test_ck01_budget();
    CALL p_test_uq01_task_position();
    CALL p_test_uq02_task_code();
    CALL p_test_rn01_project_description();
    CALL p_test_rn01_role_project();
    CALL p_test_rn01_task_project();
    CALL p_test_rn01_role_period_employee();
    CALL p_test_rn01_role_period_role();
    CALL p_test_rn01_task_period_employee();
    CALL p_test_rn01_task_period_task();
    CALL p_test_ck02_period();
    CALL p_test_rn03_task_period_overlap();
    CALL p_test_rn03_task_period_update_overlap();
    CALL p_test_rn04_subtask_same_project();
    CALL p_test_rn04_subtask_update_project();
    CALL p_test_uq03_national_id();
    SELECT * FROM test_results ORDER BY test_id;
    SELECT test_status, COUNT(*) total FROM test_results GROUP BY
test_status;
END //
DELIMITER ;

```

```
CALL p_run_tests();
```

i Info

Versión PDF disponible